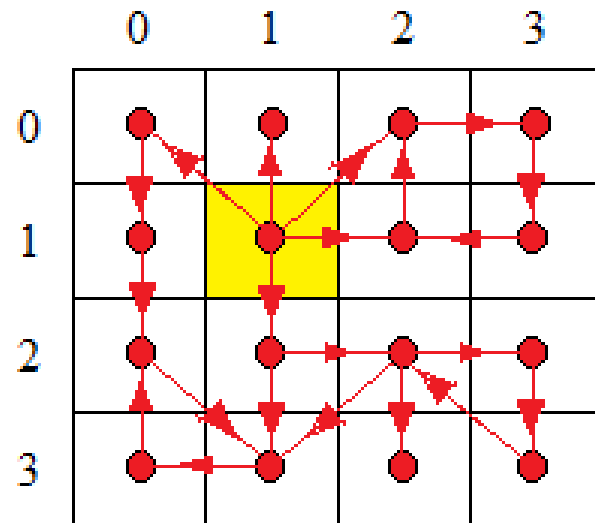


Chương 7 : Game Tree

7.1 Thuật toán *tìm theo chiều rộng trước* (BSF) trên mảng 2 chiều (TT Loan trên ma trận):

Cho ma trận G và qui các qui định chuyển từ ô (phần tử) $G[i][j]$ sang ô $G[m][n]$ như hình sau :



Biểu diễn đồ thị :

- $G[i][j]$ có giá là 1 số nguyên từ 0 đến 255. Khi đổi sang số nhị phân sẽ cho biết ô $G[i][j]$ được phép đi tới ô nào.
- Giá trị nhị phân của $G[i][j]$ có dạng : $b_7b_6...b_0$. Từ ô $G[i][j]$ chỉ được chuyển sang ô $G[m][n]$ nếu b_i trong ô đó có giá trị 1.

j			
i	b_0	b_7	b_6
	b_1		b_5
	b_2	b_3	b_4

Biểu diễn đồ thị

VD :

	j		
	0	0	0
i	1 ← 6	0	0
	1 ↘	0	0

- Các ô biên sẽ có giá trị số nguyên sao cho các ô nằm ngoài ma trận có giá trị nhị phân là 0.

VD :

	0	0	0	
	0		1	
	1	0	0	

Ngoài biên

Biên

Thuật toán BFS :

```
initQ(&Q); D[1][1]=1; push(&Q,1, 1);
while(QisEmpty(Q)==0) {
1. valueQ(Q,p,q);
2. Đổi giá trị G[p][q] sang nhị phân và lưu vào mảng B;
3. for(v=7; v>=0; v--) {
4.     if(B[v]==1) {
5.         Xác định ô (m, n) tương ứng bit v;
6.         if (D[m][n]==0) {
7.             D[m][n]=1;
8.             P[m][n].i=p; P[m][n].j=q;
9.             push(&Q, m, n);
        }
    }
} // for
10. deleteQ(&Q);
} // while
```

Kết thúc thuật toán

Thuật toán BFS : *Ô được tô màu là ô đã được duyệt.*

0. `initQ(&Q); D[1][1]=1; push(&Q,1, 1);`

`while(QisEmpty(Q)==0) {`

1. `valueQ(Q,p,q);`

2. `G[p][q] → B nhị phân;`

3. `for(v=7; v>=0; v--) {`

4. `if(B[v]==1) {`

5. `Xác định ô (m, n) tương ứng bit v;`

6. `if (D[m][n]==0) {`

7. `D[m][n]=1;`

8. `P[m][n].i=p; P[m][n].j=q;`

9. `push(&Q, m, n);`

`}`

`}`

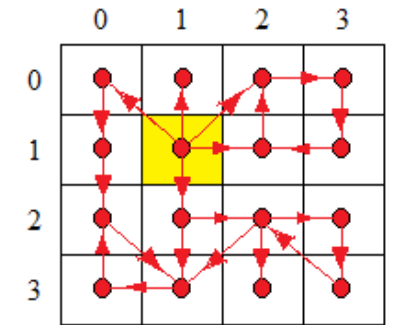
`} // for`

10. `deleteQ(&Q);`

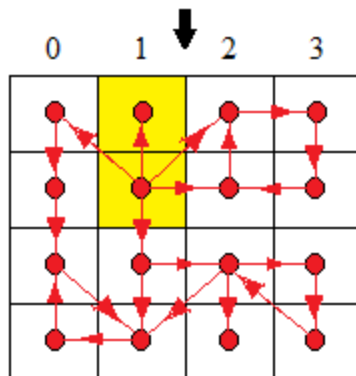
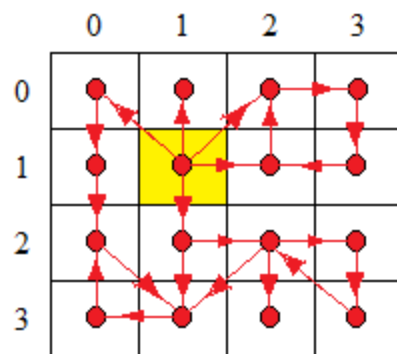
`} // while`

Kết thúc thuật toán

Q	
p	q
1	1



p	q
1	1



Thuật toán BFS : *Ô được tô màu là ô đã được duyệt.*

0. initQ(&Q); D[1][1]=1; push(&Q,1, 1);

while(QisEmpty(Q)==0) {

1. valueQ(Q,p,q);

2. $G[p][q] \rightarrow B$ nhị phân;

3. for(v=7; v>=0; v--) { **v = 7**

4. if(B[v]==1) {

5. Xác định ô (m, n) tương ứng bit v;

6. if (D[m][n]==0) {

7. D[m][n]=1;

8. P[m][n].i=p; P[m][n].j=q;

9. push(&Q, m, n);

}

}

} // for

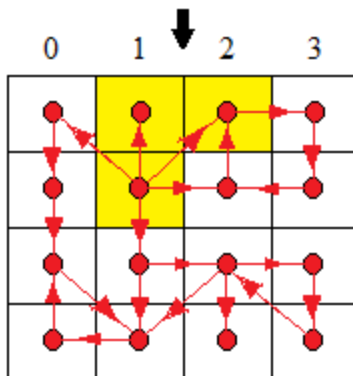
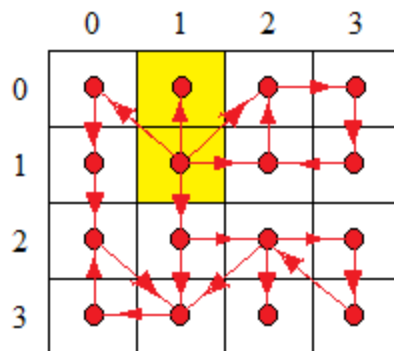
10. deleteQ(&Q);

} // while

Kết thúc thuật toán

Q	
p	q
1	1
0	1

p	q
1	1



Thuật toán BFS : *Ô được tô màu là ô đã được duyệt.*

0. initQ(&Q); D[1][1]=1; push(&Q,1, 1);

while(QisEmpty(Q)==0) {

1. valueQ(Q,p,q);

2. $G[p][q] \rightarrow B$ nhị phân;

3. for(v=7; v>=0; v--) { **v = 6**

4. if(B[v]==1) {

5. Xác định ô (m, n) tương ứng bit v;

6. if (D[m][n]==0) {

7. D[m][n]=1;

8. P[m][n].i=p; P[m][n].j=q;

9. push(&Q, m, n);

}

}

} // for

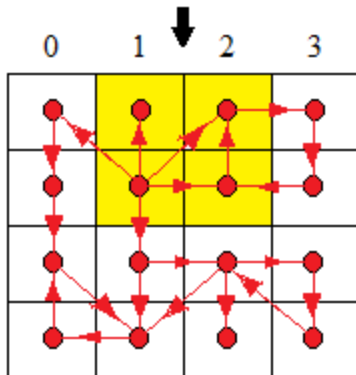
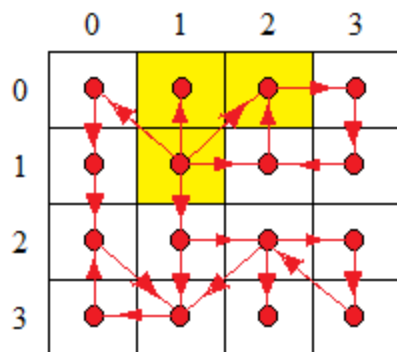
10. deleteQ(&Q);

} // while

Kết thúc thuật toán

Q	
p	q
1	1
0	1
0	2

p	q
1	1



Thuật toán BFS : *Ô được tô màu là ô đã được duyệt.*

0. initQ(&Q); D[1][1]=1; push(&Q,1, 1);

while(QisEmpty(Q)==0) {

1. valueQ(Q,p,q);

2. $G[p][q] \rightarrow B$ nhị phân;

3. for(v=7; v>=0; v--) { **v = 5**

4. if(B[v]==1) {

5. Xác định ô (m, n) tương ứng bit v;

6. if (D[m][n]==0) {

7. D[m][n]=1;

8. P[m][n].i=p; P[m][n].j=q;

9. push(&Q, m, n);

}

}

} // for

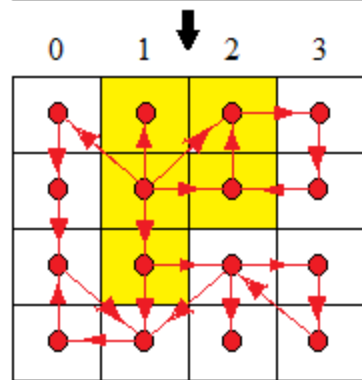
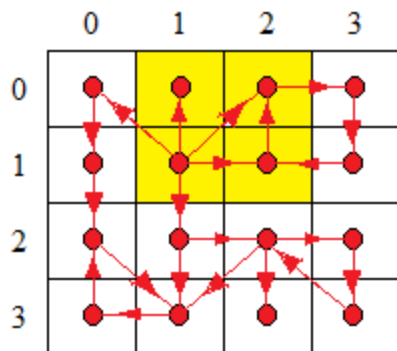
10. deleteQ(&Q);

} // while

Kết thúc thuật toán

Q	
p	q
1	1
0	1
0	2
1	2

p	q
1	1



Thuật toán BFS : *Ô được tô màu là ô đã được duyệt.*

0. initQ(&Q); D[1][1]=1; push(&Q,1, 1);

while(QisEmpty(Q)==0) {

1. valueQ(Q,p,q);

2. $G[p][q] \rightarrow B$ nhị phân;

3. for(v=7; v>=0; v--) { **v = 3**

4. if(B[v]==1) {

5. Xác định ô (m, n) tương ứng bit v;

6. if (D[m][n]==0) {

7. D[m][n]=1;

8. P[m][n].i=p; P[m][n].j=q;

9. push(&Q, m, n);

}

}

} // for

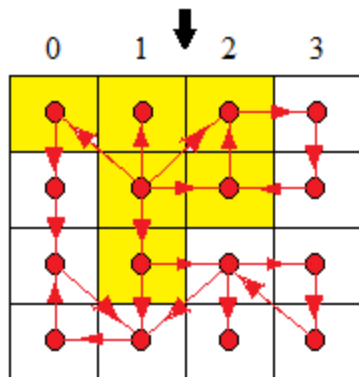
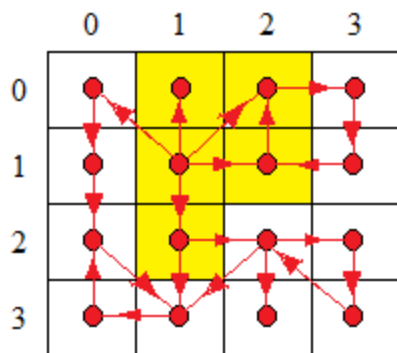
10. deleteQ(&Q);

} // while

Kết thúc thuật toán

Q	
p	q
1	1
0	1
0	2
1	2
2	1

p	q
1	1



Thuật toán BFS : *Ô được tô màu là ô đã được duyệt.*

0. initQ(&Q); D[1][1]=1; push(&Q,1, 1);

while(QisEmpty(Q)==0) {

1. valueQ(Q,p,q);

2. G[p][q] → B nhị phân;

3. for(v=7; v>=0; v--) { **v = 0**

4. if(B[v]==1) {

5. Xác định ô (m, n) tương ứng bit v;

6. if (D[m][n]==0) {

7. D[m][n]=1;

8. P[m][n].i=p; P[m][n].j=q;

9. push(&Q, m, n);

}

}

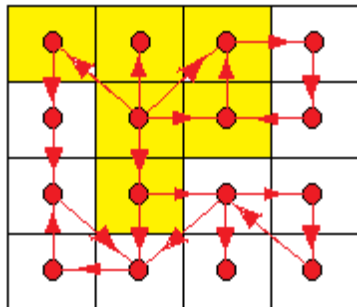
} // for

10. deleteQ(&Q);

} // while

Kết thúc thuật toán

Q	
p	q
1	1
0	1
0	2
1	2
2	1
0	0



Thuật toán BFS : *Ô được tô màu là ô đã được duyệt.*

0. `initQ(&Q); D[1][1]=1; push(&Q,1, 1);`

`while(QisEmpty(Q)==0) {`

1. `valueQ(Q,p,q);`

2. `G[p][q] → B nhị phân;`

3. `for(v=7; v>=0; v--) { v = 0`

4. `if(B[v]==1) {`

5. `Xác định ô (m, n) tương ứng bit v;`

6. `if (D[m][n]==0) {`

7. `D[m][n]=1;`

8. `P[m][n].i=p; P[m][n].j=q;`

9. `push(&Q, m, n);`

`}`

`}`

`} // for`

10. `deleteQ(&Q);`

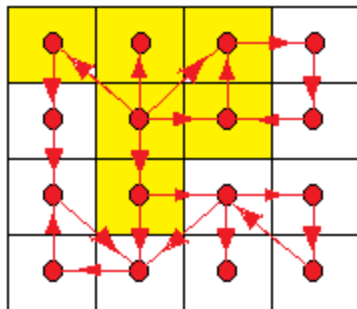
`} // while`

Kết thúc thuật toán

Q	
p	q
1	1
0	1
0	2
1	2
2	1
0	0

Xóa (1, 1), phần tử đỉnh Q.

p	q
0	1



Thuật toán BFS : *Ô được tô màu là ô đã được duyệt.*

0. initQ(&Q); D[1][1]=1; push(&Q,1, 1);

while(QisEmpty(Q)==0) {

1. valueQ(Q,p,q);

2. G[p][q] → B nhị phân;

3. for(v=7; v>=0; v--) { **Không có v**

4. if(B[v]==1) {

5. Xác định ô (m, n) tương ứng bit v;

6. if (D[m][n]==0) {

7. D[m][n]=1;

8. P[m][n].i=p; P[m][n].j=q;

9. push(&Q, m, n);

}

}

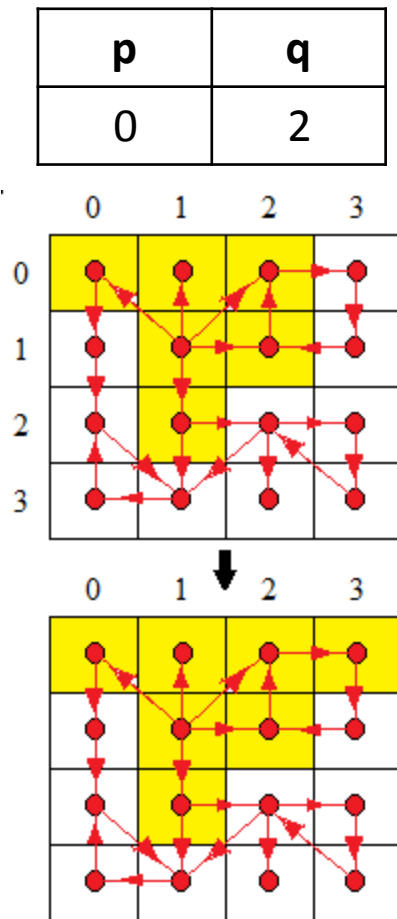
} // for

10. deleteQ(&Q); *Xóa phần tử (0, 1) trong Q*

} // while

Kết thúc thuật toán

Q	
p	q
0	1
0	2
1	2
2	1
0	0



Thuật toán BFS : *Ô được tô màu là ô đã được duyệt.*

0. initQ(&Q); D[1][1]=1; push(&Q,1, 1);
while(QisEmpty(Q)==0) {

1. valueQ(Q,p,q);

2. G[p][q] → B nhị phân;

3. for(v=7; v>=0; v--) { **v = 5**

4. if(B[v]==1) {

5. Xác định ô (m, n) tương ứng bit v;

6. if (D[m][n]==0) {

7. D[m][n]=1;

8. P[m][n].i=p; P[m][n].j=q;

9. push(&Q, m, n);

}

}

} // for

10. deleteQ(&Q);

} // while

Kết thúc thuật toán

Q	
p	q
0	2
1	2
2	1
0	0
0	3

Sinh viên tiếp tục cho đến khi Q rỗng.

Kết quả mảng P :

$P[0,0] = (1, 1)$, $P[0,1] = (1, 1)$, $P[0,2] = (1, 1)$, $P[0,3] = (0, 2)$,
 $P[1,0] = (0, 0)$, $P[1,1] = (-1, -1)$, $P[1,2] = (1, 1)$, $P[1,3] = (0, 3)$,
 $P[2,0] = (1, 0)$, $P[2,1] = (1, 1)$, $P[2,2] = (2, 1)$, $P[2,3] = (2, 2)$,
 $P[3,0] = (3, 1)$, $P[3,1] = (2, 1)$, $P[3,2] = (2, 2)$, $P[3,3] = (2, 3)$,

Bài toán : Tìm đường đi từ gốc = (1,1) tới (3,2).

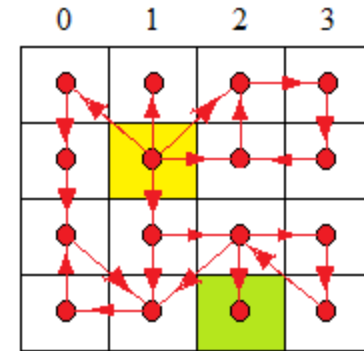
- $P[3,2] = (2, 2)$

- $P[2,2] = (2, 1)$

- $P[2,1] = (1, 1)$

- $P[1,1] = (-1, -1)$, STOP.

Kết quả : $(1,1) \rightarrow (2,1) \rightarrow (2,2) \rightarrow (3,2)$.



Viết chương trình xác định đường đi từ (i,j) đến (m,n) (Bài tập).

- Có thể thể không có đường đi từ (i,j) đến (m,n).
- Giả sử (0,2) không thể chuyển sang (0,3).
- Thực hiện thuật toán, gốc (1,1).
Quan sát P.

7.2 Game tree:

Bài toán dồn sỏi :

- Cho k và n là 2 số nguyên dương, có 2^k viên sỏi, được phân bố trong n đồng.
- Người ta cần san sẻ (theo quy tắc dưới đây) lượng sỏi từ các đồng để dồn sỏi trở về một đồng.
- Quy tắc san sỏi như sau: mỗi lần san áp dụng cho hai đồng sỏi, giả sử rằng một đồng có a viên sỏi và còn đồng kia có b viên sỏi (không giảm tổng quát, giả thiết $a \geq b$) thì cho phép san (thay đổi số lượng từ hai đồng) như sau: đồng a viên (đồng có số sỏi không thua đồng còn lại) sẽ là $a - b$ viên, còn đồng b trở thành $2 \cdot b$ viên.

Hãy tìm cách san sỏi để cuối cùng còn 1 đồng sỏi với 2^k viên.

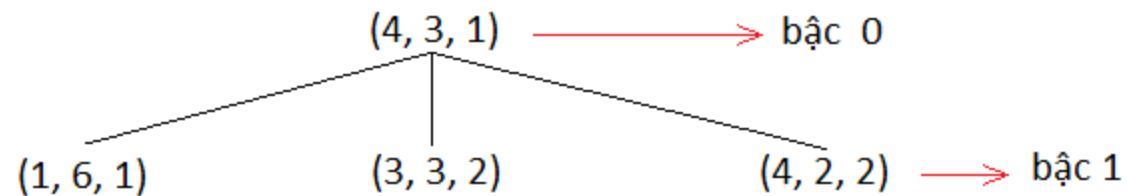
7.2 Game tree:

Bài toán dồn sỏi : $k=3, n=3$. Đồng 1 có 4 viên sỏi, đồng 2 có 3 viên sỏi, đồng 3 có 1 viên sỏi. Biểu diễn bài toán bởi một bộ $(4, 3, 1)$, được gọi là trạng thái của bài toán.

Từ trạng thái $(4, 3, 1)$ có thể chuyển sang các trạng thái sau :

- Hai đồng được chọn là $\{1, 2\}$, dồn đồng 1 sang đồng 2, ta có trạng thái $(1, 6, 1)$,
- Hai đồng được chọn là $\{1, 3\}$, dồn đồng 1 sang đồng 3, ta có trạng thái $(3, 3, 2)$,
- Hai đồng được chọn là $\{2, 3\}$, dồn đồng 2 sang đồng 3, ta có trạng thái $(4, 2, 2)$.

Biểu diễn ở dạng “cây” :

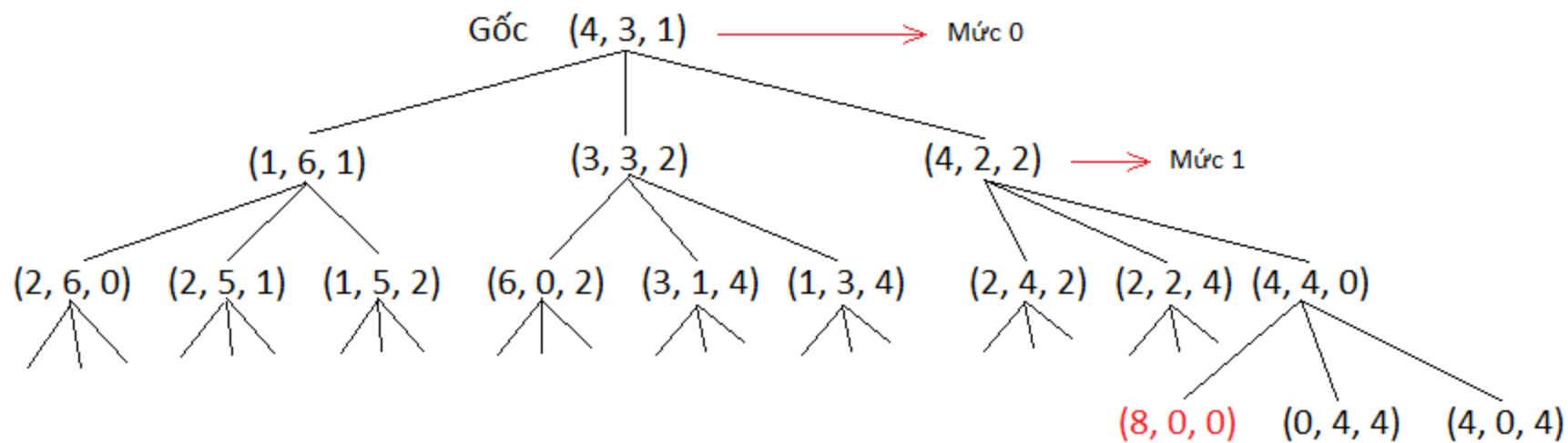


Ta tiếp tục giống như trên cho các trạng thái ở bậc 1. Ta được các trạng thái bậc 2, ...

Lưu ý :

- Cặp $\{1, 2\}$ và cặp $\{2, 1\}$ được xem là như nhau,
- Nếu giá trị của cặp $\{i, j\}$, $i < j$, là bằng nhau thì dồn sỏi về i .

Bài toán 1 (Bài toán dồn sỏi) : Các cặp được chọn lần lượt là $\{1, 2\}$, $\{1, 3\}$, $\{2, 3\}$. Cây trạng thái của bài toán (Game tree) :



Cách thực hiện là : Gốc $\rightarrow \{2, 3\} \rightarrow \{2, 3\} \rightarrow \{1, 2\}$

Thuật toán dồn sỏi : Dùng thuật toán BFS

Thuật toán dồn sỏi : Dùng thuật toán BFS (xem file *ThuatToan-DonSoi-Chuong-5*)

Bước 1 :

QUEUE Q;

int *P[10]; // Lưu dấu vết

int DongSoi[3]={4, 3, 1};

int win=0;

initQ(&Q);

1. push(&Q,DongSoi[0], DongSoi[1], DongSoi[2], 0, 0);

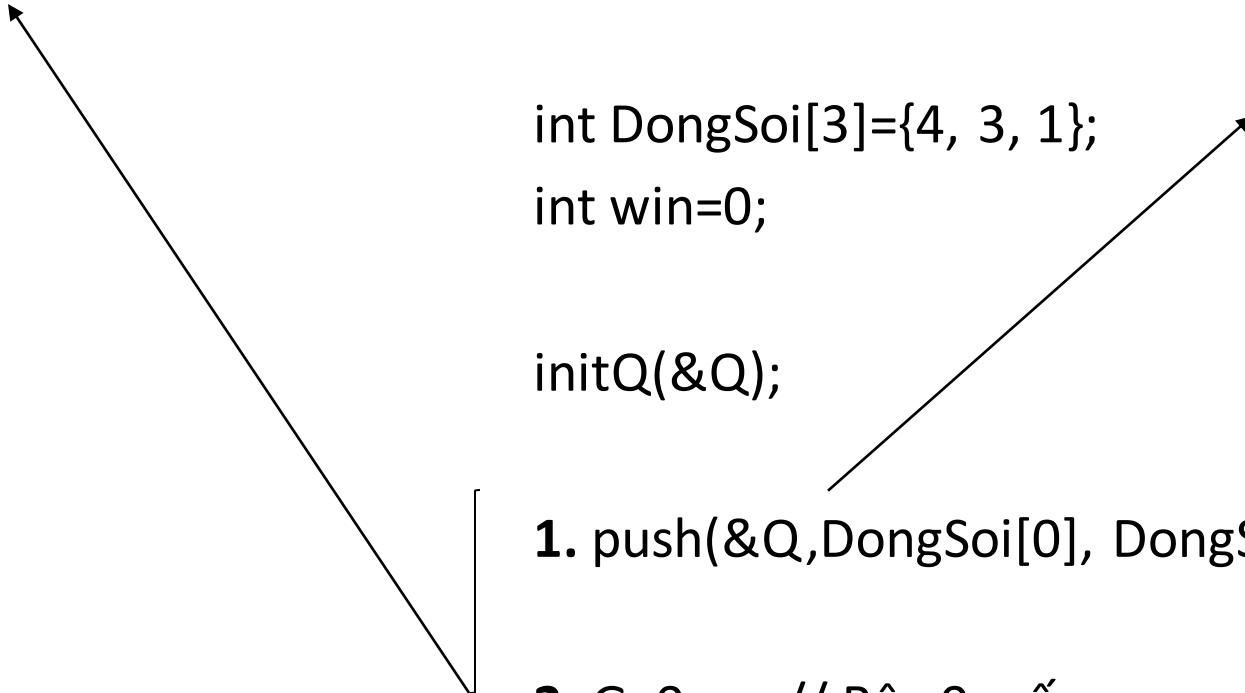
2. G=0; // Bậc 0, gốc

3. P[G]=(int*)malloc(sizeof(int));

4. P[G][0]= -1;

(4, 3, 1) P[0][0] -1

Q				
i	j	k	id	G
4	3	1	0	0



Thuật toán dồn sỏi : Dùng thuật toán BFS (xem file *ThuatToan-DonSoi-Chuong-5*)

Bước 2 :

1. $id=0$; $G++$; $NUM=3^G$; $P[G]$ = Cấp phát NUM vùng nhớ;

while($win==0$)

{

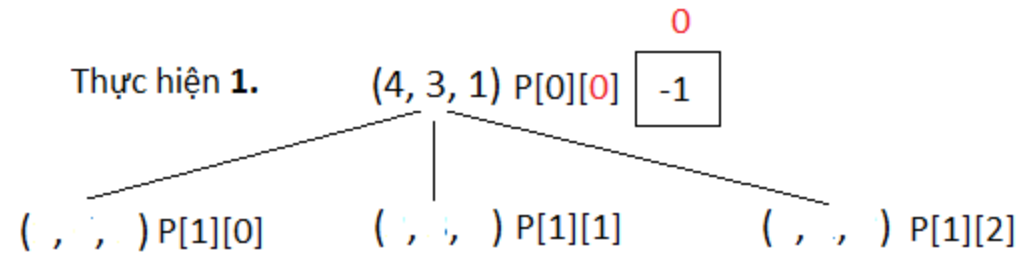
2. $valueQ(Q,i,j,k,id_p,G_p)$;

i	j	k	id_p	G_p
4	3	1	0	0

Q				
i	j	k	id	G
4	3	1	0	0

...

}// while



Thuật toán dồn sỏi : Dùng thuật toán BFS (xem file *ThuatToan-DonSoi-Chuong-5*)

Bước 2 :

```
while(win==0)
{
3. if((i==8) || (j==8) || (k==8)) {
        win=1;
        break;
    }
4. if(id==NUM){
5.   id=0; G++;
6.   NUM=3^G; P[G]= Cấp phát NUM vùng nhớ
    }
...
} // while
```

i	j	k	id_p	G_p
4	3	1	0	0

Q				
i	j	k	id	G
4	3	1	0	0

Thuật toán dồn sỏi : Dùng thuật toán BFS (xem file *ThuatToan-DonSoi-Chuong-5*)

Bước 2 :

1. $id=0$; $G++$; $NUM=3^G$; $P[G]=$ Cấp phát NUM vùng nhớ;

while($win==0$)

{

7. for($v=1$; $v \leq n$; $v++$) {

8. switch(v) {

case 1: // Chọn tập {1,2}

if ($i > j$) push(&Q, $i-j$, $2*j$, k , id , G);

else if($i < j$) push(&Q, $2*i$, $j-i$, k , id , G);

else push(&Q, $2*i$, 0 , k , id , G);

break;

}// switch

9. $P[G][id]=id_p$;

10. $id++$; // $id = 3 = NUM$

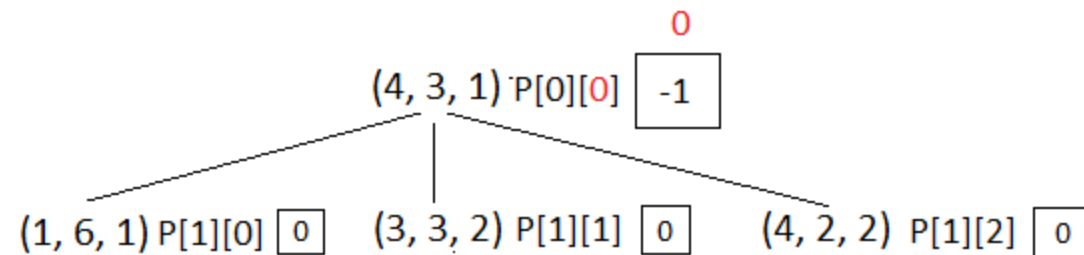
}// for

11. deleteQ(&Q)

}// while

i	j	k	id_p	G_p
4	3	1	0	0

Q				
i	j	k	id	G
4	3	1	0	0
1	6	1	0	1
3	3	2	1	1
4	2	2	2	1



Thuật toán dồn sỏi : Dùng thuật toán BFS (xem file *ThuatToan-DonSoi-Chuong-5*)

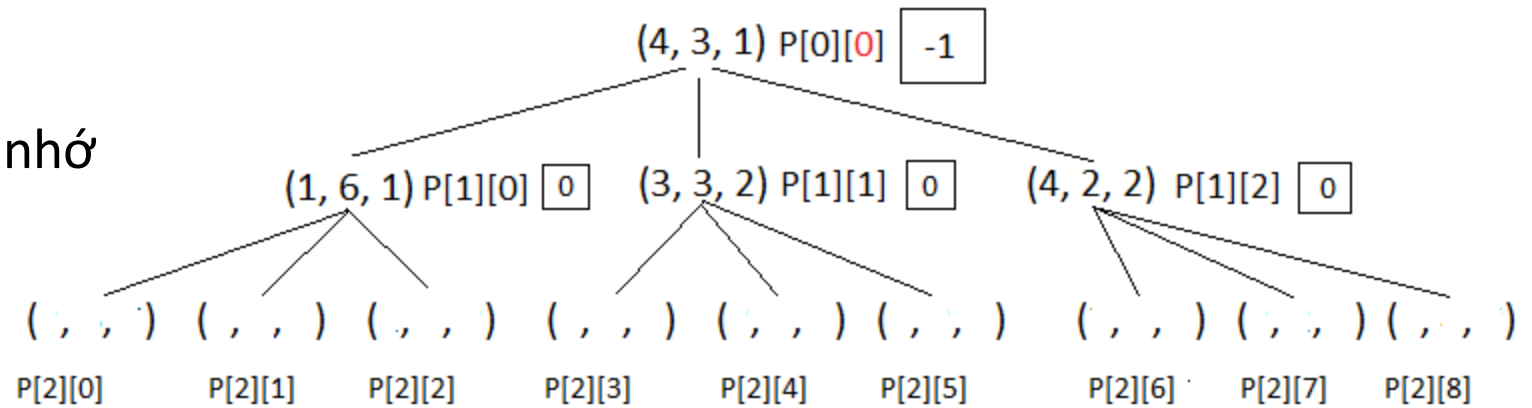
Bước 2 :

```

while(win==0)
{
2. valueQ(Q,i, j, k, id_p, G_p);
3. if((i==8) || (j==8) || (k==8)) {
    win=1;
    break;
}
4. if(id==NUM){
5.   id=0; G++;
6.   NUM=3^G;
   P[G]= Cấp phát NUM vùng nhớ
}
...
} // while
    
```

i	j	k	id_p	G_p
1	6	1	0	1

Q				
i	j	k	id	G
4	3	1	0	0
1	6	1	0	1
3	3	2	1	1
4	2	2	2	1



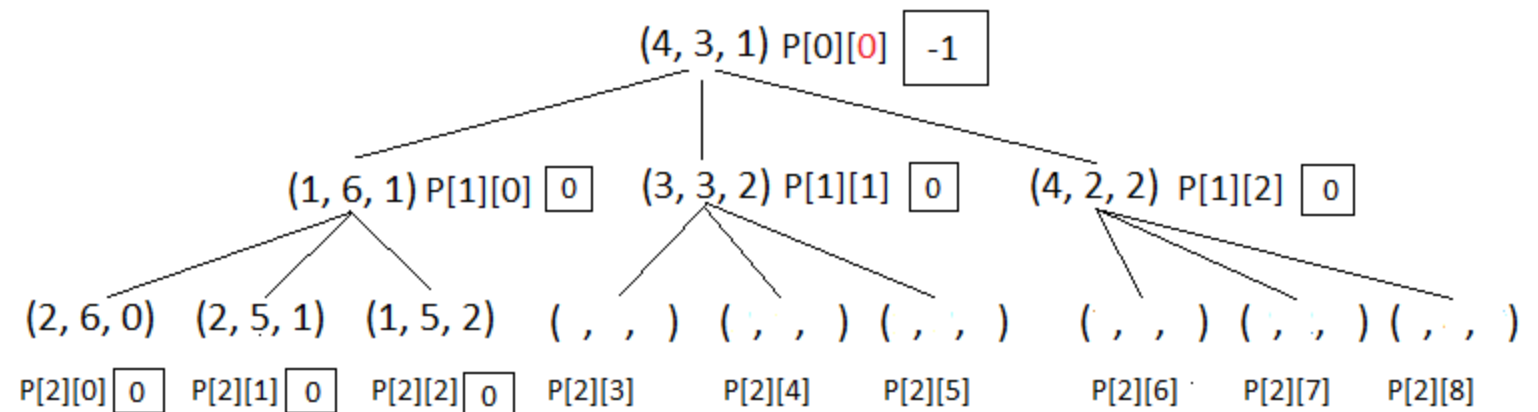
Thuật toán dồn sỏi : Dùng thuật toán BFS (xem file *ThuatToan-DonSoi-Chuong-5*)

Bước 2 :

```
while(win==0)
{
7. for(v=1 ; v<= n; v++)
{
8. switch( v ) {...}
9. P[G][id]=id_p;
10. id++;
} // for → id = 3;
11. deleteQ(&Q);
} // while
```

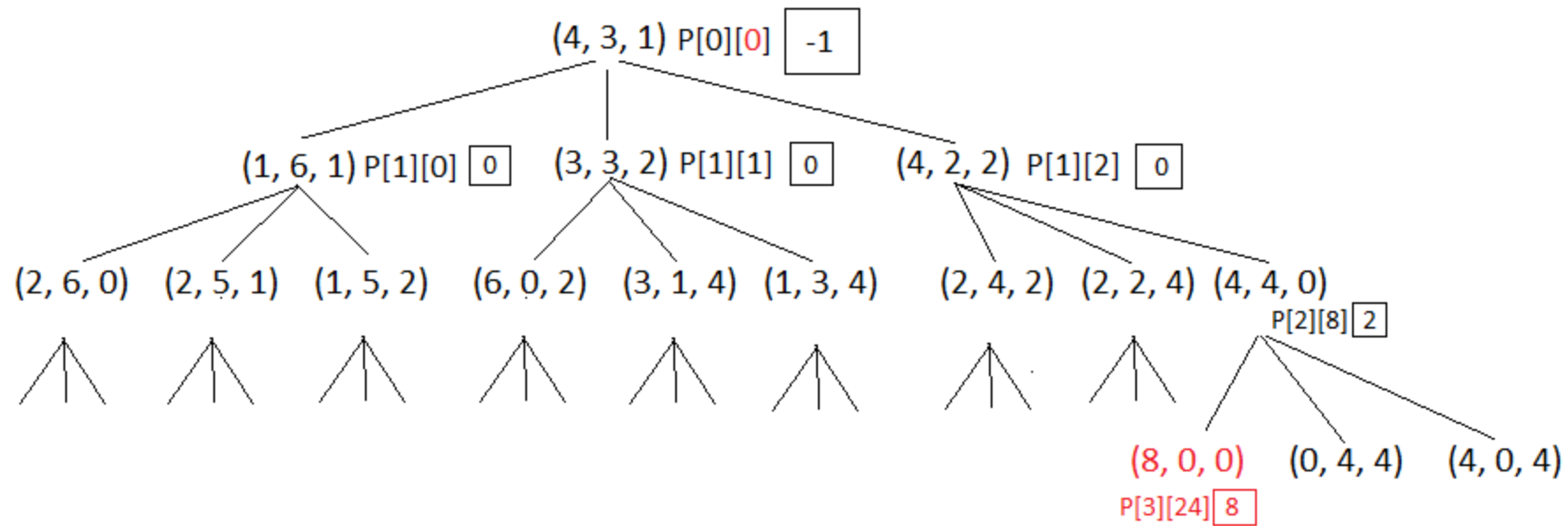
i	j	k	id_p	G_p
1	6	1	0	1

Q				
i	j	k	id	G
4	3	1	0	0
1	6	1	0	1
3	3	2	1	1
4	2	2	2	1



Thuật toán dồn sỏi : Dùng thuật toán BFS (xem file *ThuatToan-DonSoi-Chuong-5*)

Kết quả



Bài toán 2 :

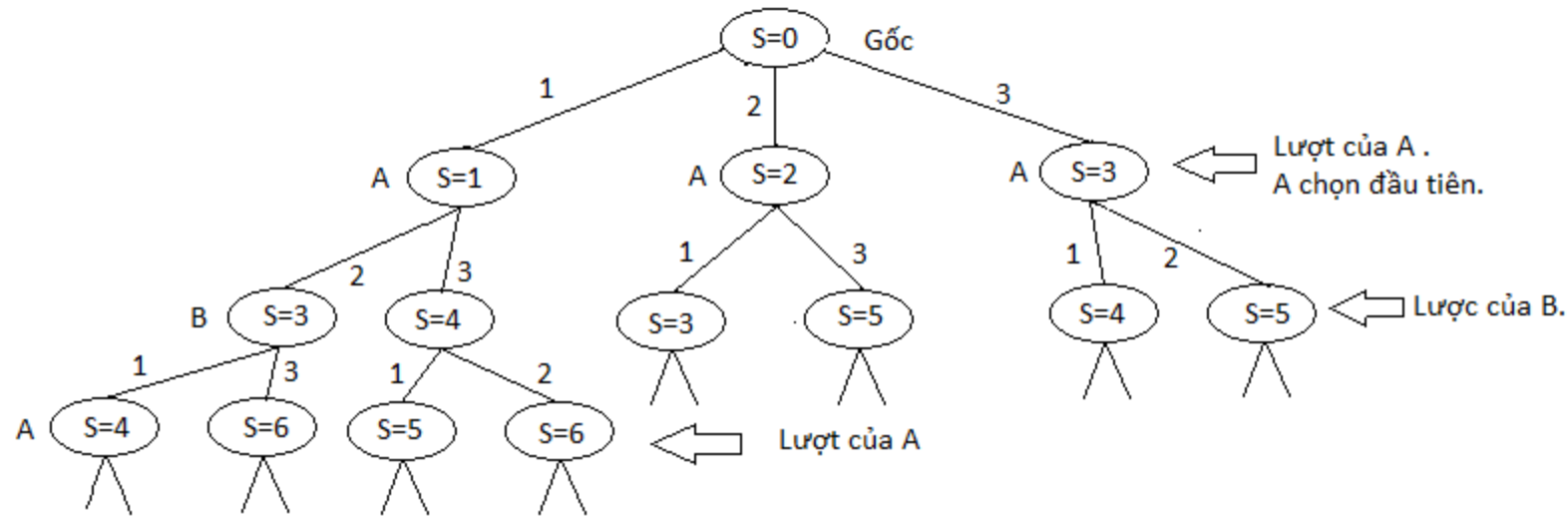
- Cho các số 1, 2, 3 và tổng S ban đầu bằng 0.
- Có 2 người chơi lần lượt chọn một trong các số đã cho theo nguyên tắc không được chọn số mà người kia vừa chọn trước đó, mỗi lần ai chọn đều cộng dồn vào tổng S.

Ví dụ:

- Ban đầu người A chọn 2 vậy $S=0+2=2$.
- Tiếp theo người B chỉ được chọn 1, 3 (không được chọn 2), ví dụ chọn 1, vậy $S=2+1=3$.
- Tiếp theo A chỉ được chọn 2, 3 (được chọn lại 2 nhưng không được chọn 1), ví dụ chọn 3. Vậy $S=3+3=6$.
- Tiếp theo B chỉ được chọn 1, 2 (không được chọn 3 vì A mới vừa chọn 3), ví dụ chọn 2. Vậy $S=6+2=8....$

Ai làm cho tổng S lớn hơn 7 là thua.

Cây trạng thái :



- Giả sử A chọn trước,
- Từ cây trạng thái , có thể quyết định được A sẽ thắng , không phụ thuộc vào việc chọn của B.

Thuật toán : Dùng thuật toán BFS. (Bài tập)